

Étude de l'apprentissage des abeilles par modèles d'apprentissage par renforcement.

Rémi AIRIAU

Licence Informatique – INXPP61A1
Enseignant : Vincent DUGAT
Encadrante : Emanuelle CLAEYS | IRIT
Cliente : Elena KERJEAN | CRCA

Dernière modification : 27/05/2025



UE Bureau d'études / stage

Résumé

Ce document présente le projet et son domaine d'application. Il inclut aussi une introduction à l'apprentissage par renforcement. Une autre section tient à jour l'avancement courant, ainsi que les améliorations possibles à long terme. Ensuite vient une documentation plus technique (à l'image de la documentation d'une librairie). Ainsi, on y trouve comment est structuré le code, comment ont été implémentées certaines fonctionnalités et enfin quelques difficultés rencontrées. Il est donc particulièrement destiné à toute personne souhaitant comprendre, modifier ou réutiliser l'application : enseignant-e-s, chercheur-e-s, ou étudiant-e-s.

Mots-clefs

Apprentissage par renforcement ; Biologie ; R ; Shiny ; Application Web ;

Table des matières

I	Introduction	4
I.1	Présentation du sujet	4
I.2	Introduction à l'apprentissage par renforcement	4
I.3	Markov Decision Process and Multi-Armed Bandit	4
I.3.1	MDP	4
I.3.2	MAB	5
I.3.3	Ressources pour une approche plus formelle	5
I.3.3.1	Liens Wikipédia	5
I.3.3.2	Vidéos sur les MAB	5
I.3.3.3	Cours sur le RL	5
I.3.3.4	Lien github de banditWithR	5
I.4	Description de l'expérience	6
II	Format de la première expérience	7
II.1	Stimuli et récompenses	7
II.1.1	Encodage des stimuli	7
II.1.2	Visualisation des stimuli	8
II.2	Formatage des données	9
III	Cahier des charges	10
III.1	Fonctionnalités maintenues	10
III.2	Fonctionnalités reportées	10
III.2.1	Liste des fonctionnalités reportées	10
III.2.2	Justifications	11
III.3	Nouvelles perspectives d'améliorations	11
III.4	Travail réalisé	11
III.4.1	Liste des tâches techniques effectuées	11
III.4.2	Aperçu du prototype initial et de l'application	12
IV	Documentation de l'application	13
IV.1	Structure du projet	13
IV.1.1	BeesRLWithR	13
IV.1.2	src/main	13
IV.1.3	Installation	13
IV.2	Notions fondamentales de Shiny utilisées dans le projet	14
IV.2.1	inputs et outputs	14
IV.2.2	<code>reactive()</code> , <code>observe()</code> et <code>observeEvent()</code>	14
IV.2.3	Les modules	15
IV.3	Commentaires sur l'implémentation	17
IV.3.1	Séparation des fonctionnalités	17
IV.3.1.1	ui.R	17
IV.3.1.2	server.R	18
IV.3.1.3	Fichiers autres	20
IV.3.2	Les environnements	20
IV.3.3	Importation des données	22
IV.3.3.1	Lecture d'un fichier	23
IV.3.3.2	Détecter l'expérience	24
IV.3.3.3	Vérification du format	24
IV.3.3.4	Traitement initial des données	25
IV.3.4	Définition des modèles	27

IV.3.5 Définition des tests statistiques	30
IV.3.5.1 Divergence de Kullback-Leibler	30
IV.3.5.2 Distance de Wasserstein	30
IV.3.5.3 Implémentations de la divergence KL et de la distance de Wasserstein	31
IV.3.5.4 Visualisation des tests statistiques	32
V Contact	34
A Bibliographie	35
B Implémentation du cours sur les MDP	36
C Implémentation des vidéos sur les MAB	40

I – Introduction

I.1 Présentation du sujet

L'objectif de ce projet est d'étudier le processus d'apprentissage des abeilles grâce à des modèles d'apprentissage par renforcement. Nous devons pour cela développer un outil interactif (application web de type dashboard), en utilisant principalement la technologie [ShinyR](#), afin de permettre à un utilisateur, à partir d'un jeu de données importé, d'obtenir des indicateurs sur la capacité d'une ou plusieurs abeilles à répondre à un problème d'apprentissage séquentiel. Il serait souhaitable à terme de déployer l'application sur [shinyapps.io](#) dans le but de rendre son utilisation plus accessible.

Nous utiliserons des algorithmes d'apprentissage par renforcement (ici des algorithmes de bandit) préalablement développés par Emmanuelle CLAEYS lors de sa thèse et nous intégrerons peut-être également de nouvelles méthodes.

Ces algorithmes proviennent de la librairie *banditWithR* :

<https://github.com/manuclaeys/banditWithR>

Vous trouverez également le dépôt github du projet *BeesRLWithR* à cette adresse :

<https://github.com/Ravemi987/BeesRLWithR>

I.2 Introduction à l'apprentissage par renforcement

Comme mentionné ci-dessus, nous utilisons des modèles d'apprentissage par renforcement pour comparer le comportement d'une abeille avec une solution algorithmique optimale au même problème. Voici donc une courte introduction à l'apprentissage par renforcement. Vous trouverez dans cette partie différentes ressources que nous avons consultées pour mieux appréhender le concept et pouvoir ensuite travailler sur le projet.

De manière la plus générale possible, l'apprentissage par renforcement est un sous-domaine de l'intelligence artificielle et plus précisément de l'apprentissage automatique. Un agent autonome (ex : humain, animal, robot...), placé dans un environnement expérimental, doit choisir itérativement entre plusieurs actions, appelées "bras". Au travers de ses actions, l'agent va recevoir une récompense positive ou négative, selon une distribution de probabilité qui lui est inconnue. L'agent doit alors apprendre progressivement quelle option est la meilleure en établissant une stratégie ou politique dans le but de maximiser ses récompenses positives et minimiser son regret. Cette stratégie oscille entre exploration (tester une nouvelle action pour espérer gagner plus) et exploitation (choisir l'action qui récompense le mieux jusqu'à présent).

I.3 Markov Decision Process and Multi-Armed Bandit

Plusieurs types de modèles sont utilisés en apprentissage par renforcement. Nous allons nous baser sur les MDP (*Markov Decision Process*) ainsi que les MAB (*Multi-Armed Bandit*)

I.3.1 MDP

Pour faire simple, un MDP est un automate défini par :

- S - un ensemble d'états
- A - un ensemble d'actions

- P - une fonction de transition qui à un état s et une action a , associe l'état s' dans lequel on arrive en effectuant l'action a depuis l'état s .
- R - une fonction de récompense qui à un état s et une action a , associe la récompense obtenue par l'automate après avoir effectué l'action a .

La spécificité d'un MDP est que la probabilité de transition d'un état vers un nouvel état ne dépend que de l'état précédent. Le but pour l'automate est de trouver une politique optimale, c'est-à-dire la meilleure action à prendre à partir de n'importe quel état pour maximiser le gain.

I.3.2 MAB

Le Multi-Armed Bandit (MAB) est un problème classique en apprentissage par renforcement où un agent doit choisir parmi K actions (ou "bras") afin de maximiser sa récompense totale. Contrairement aux MDP où l'agent se déplace entre des états, ici il reste toujours dans le même état et doit juste choisir la meilleure action au fil du temps. C'est un cas particulier de MDP.

Définition du problème :

- A chaque instant, l'agent **choisit un bras** parmi un ensemble d'actions.
- **Chaque bras suit une distribution de probabilité** inconnue, qui représente les récompenses obtenues en jouant ce bras.
- L'agent reçoit une **récompense** qui suit cette distribution
- **Objectif** : choisir les bras de manière optimale pour maximiser la récompense.

I.3.3 Ressources pour une approche plus formelle

I.3.3.1 Liens Wikipédia

Les pages Wikipédia expliquent bien les concepts et ne sont pas trop compliquées mathématiquement parlant. Elles peuvent être lues en entier :

- [Processus de décision markovien \(MDP\)](#)
- [Problème du bandit manchot \(MAB\)](#)

I.3.3.2 Vidéos sur les MAB

Ces deux vidéos expliquent également très bien (en anglais) les MAB. Vous trouverez en annexe un code permettant d'implémenter le problème traité. [B](#)

[Multi-Armed Bandit : Data Science Concepts](#)

[Best Multi-Armed Bandit Strategy? \(feat: UCB Method\)](#)

I.3.3.3 Cours sur le RL

Ensuite, vous pouvez consulter le cours de Mme Emmanuelle Claeys sur l'apprentissage par renforcement : [Introduction au RL et aux bandits multi-armés](#)

De même, vous trouverez en annexe une implémentation des exemples : *Value Iteration : a toy example*, *Policy Iteration : a toy example* et *Q-learning : a toy example*. [C](#)

I.3.3.4 Lien github de banditWithR

Enfin, il est nécessaire de comprendre les algorithmes UCB, LinUCB, Epsilon Greedy et Uniform du dépôt github de la librairie banditWithR, ainsi que les fonctions auxiliaires associées : [ici](#).

I.4 Description de l'expérience

Dans notre cas, l'agent est une abeille. Au départ, une abeille appartenant à une ruche est attirée dans une boîte fermée contenant deux stimuli (par exemple des formes visuelles). C'est notre environnement. Celui-ci procure à l'abeille une récompense qui dépend du stimulus choisi. Cette récompense est soit positive (du sucre), soit négative (une substance amère). Ensuite, l'abeille retourne à la ruche, ce qui valide une itération (un essai) de l'expérience. Les formes servant de stimuli sont régulièrement substituées par des versions légèrement modifiées (couleur, taille...). Puis, le chercheur prend note de la date et l'heure du relevé, du numéro de l'abeille, du nombre d'essais qu'elle a réalisés, du nombre de succès, d'échecs, etc. L'objectif étant d'analyser les données pour savoir si son degré de satisfaction (sa récompense) augmente au cours du temps, et de comparer sa stratégie à celle de modèles connus d'apprentissage par renforcement.

Il peut y avoir plusieurs types d'expériences qui diffèrent légèrement dans le principe, mais très peu, voire pas du tout, au niveau du traitement des données. Par exemple, une autre expérience permet à l'abeille de choisir un stimulus plusieurs fois (accumuler plusieurs essais), ou encore utilise des couleurs plutôt que des formes. Vous trouverez ci-dessous une image plus parlante de la première expérience décrite. [I.1](#)

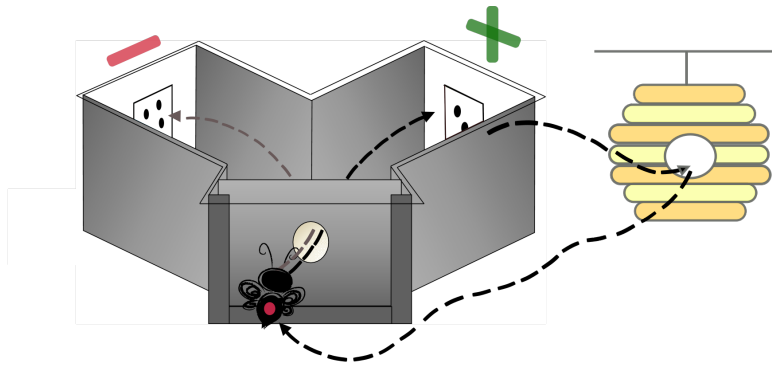


Figure 1 : Schematic of experimental set-up : Y-maze. Free-flying honeybees are recruited from a feeder, trained to enter the Y-maze and marked by a color dot. When the bee lands on a stimulus, it receives either a sugar reward or a quinine (bitter substance) punishment, depending on its choice. The bee will then travel back and forth between the experimental device and its hive to deliver the collected sugar solution.

FIGURE I.1 –
Schéma de l'expérience

II – Format de la première expérience

Comme mentionné rapidement plus haut, cette application doit être capable de s'adapter à plusieurs types d'expériences assez proches. En général, peu de modifications sont nécessaires dans le code pour supporter une nouvelle expérience, ce que nous verrons plus tard. Pour présenter le format des données, nous nous appuyerons sur la première expérience. Si besoin, relire la description de l'expérience : [I.4](#).

II.1 Stimuli et récompenses

En réalité, l'abeille reçoit soit du sucre, soit une substance amère. D'un point de vue numérique, nous lui attribuerons systématiquement une récompense, mais la récompense positive est plus élevée que la récompense négative. Cette récompense quantitative est représentée par le nombre de figures visuelles distinctes sur le stimulus. Ici ce sont des formes de 4 ou 2 cercles noirs. On dira que **l'abeille est toujours récompensée à 4**. Cela signifie qu'elle obtient la récompense si elle choisit le côté avec le stimulus comportant 4 cercles noirs, mais **les schémas sont régulièrement changés**. Voir figure [II.4](#).

II.1.1 Encodage des stimuli

Conceptuellement, il y a **18** schémas de stimuli possibles, pour **9** paires. En effet, à chaque paire correspond deux schémas : celui pour lequel l'abeille est récompensée, et celui pour lequel elle ne l'est pas. Une paire est utilisée pour 1 essai de l'abeille. Les paires sont numérotées de **1** à **9**. Ces paires peuvent aussi être regroupées en **3** classes :

- Paires n°1, 2, 3 -> classe A
- Paires n°4, 5, 6 -> classe B
- Paires n°7, 8, 9 -> classe C

On peut donc encoder les schémas de la manière suivante :

Rd-{class}-{reward}-{pair % 3 + 1}

- classe A :

	Paire 1	Paire 2	Paire 3
Récompensée	Rd-a-4-1	Rd-a-4-2	Rd-a-4-3
Non récompensée	Rd-a-2-1	Rd-a-2-1	Rd-a-2-3

TABLE II.1 – Encodage des stimuli de la classe A

- classe B :

	Paire 4	Paire 5	Paire 6
Récompensée	Rd-b-4-1	Rd-b-4-2	Rd-b-4-3
Non récompensée	Rd-b-2-1	Rd-b-2-1	Rd-b-2-3

TABLE II.2 – Encodage des stimuli de la classe B

- classe C :

	Paire 7	Paire 8	Paire 9
Récompensée	Rd-c-4-1	Rd-c-4-2	Rd-c-4-3
Non récompensée	Rd-c-2-1	Rd-c-2-1	Rd-c-2-3

TABLE II.3 – Encodage des stimuli de la classe C

II.1.2 Visualisation des stimuli

Ci-dessous, un aperçu des stimuli utilisés pour les 3 paires de la classe A.

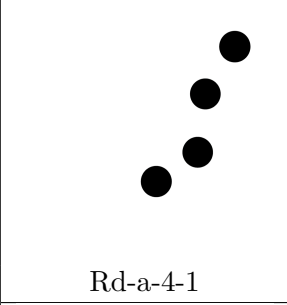
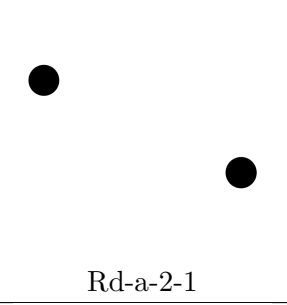
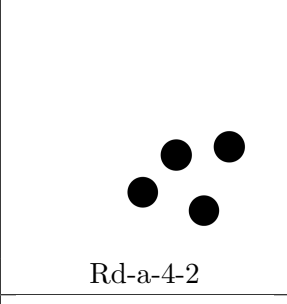
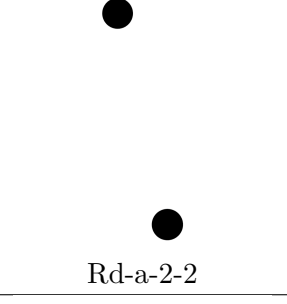
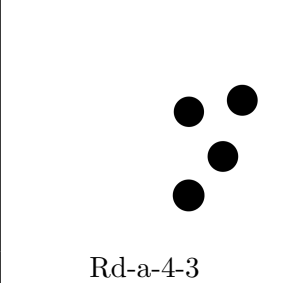
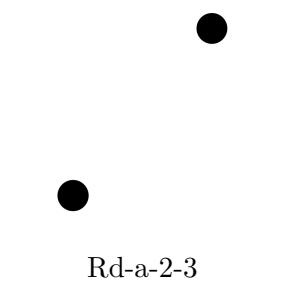
Récompensée	Non récompensée
 Rd-a-4-1	 Rd-a-2-1
 Rd-a-4-2	 Rd-a-2-2
 Rd-a-4-3	 Rd-a-2-3

TABLE II.4 – Schémas des stimuli de la classe A

II.2 Formatage des données

Voici comment sont fournies les données :

id	trial	side	pair	correct
2	1	D	1	0
2	2	D	7	0
2	3	G	2	0
2	4	D	7	0
2	5	D	6	0
2	6	G	3	0
2	7	G	4	1
2	8	D	7	0
2	9	G	7	0
2	10	D	3	1
2	11	G	9	0
2	12	G	3	0
2	13	D	3	1
2	14	G	7	0
2	15	G	2	0

TABLE II.5 – Tableau des données d’essais

- **Colonne `id`** : c’est le numéro de l’abeille. Comme elles sont triées, seules certaines données de l’abeille 2 sont présentes dans l’exemple ci-dessus.
- **Colonne `trial`** : c’est le numéro d’essai de l’abeille. Pour **cette** expérience, chaque abeille réalise **40** essais.
- **Colonne `side`** : c’est le côté **récompensé**. Comme on l’a vu plus haut, c’est donc le côté récompensé à **4**.
- **Colonne `pair`** : c’est le numéro de la paire utilisée comme stimulus pour ce numéro d’essai (entre 1 et 9)
- **Colonne `correct`** : vaut 1 si l’abeille a effectivement choisi le côté récompensé à 4, 0 si elle a choisi le côté récompensé à 2.

Exemple :

Pour la première ligne : l’abeille numéro 2 réalise son premier aller-retour entre la ruche et le "maze" [I.1](#). La bonne récompense est du côté droit (**D**). La substance est déposée à proximité du stimulus **Rd-a-4-1**. Malheureusement, l’abeille se trompe, elle part du côté gauche (**G**) (puisque **’correct’** est à 0).

On remarque que le format est donc assez strict. Il faudra dans un premier temps le vérifier lorsque de nouvelles données sont importées dans l’application. Ensuite, il est nécessaire de réaliser un prétraitement avant de les exploiter, et ce, afin de faciliter leur accès. Ces informations suffisent à déduire le côté choisi, le côté précédemment choisi, la valeur de la récompense, celle du regret, etc...

III – Cahier des charges

Ayant un peu sous-estimé au départ la quantité de travail à fournir, nous allons légèrement diminuer le cahier des charges en laissant de côté temporairement les fonctionnalités les moins importantes ou les moins intéressantes. Ce travail étant aussi utilisé pour aider à la rédaction d'une thèse en biologie, nous avons dû nous adapter à l'arrivée de nouvelles données, de nouveaux formats, etc... En d'autres termes, vous trouverez ci-dessous la liste des fonctionnalités prioritaires, puis celles reportées qui faisaient partie intégrante du cahier des charges initial (mais qui pourront être ré-introduites plus tard. Enfin, seront listées les nouvelles fonctionnalités souhaitées et le travail réalisé jusqu'à présent.

III.1 Fonctionnalités maintenues

- L'application doit être structurée en 4 sections principales : informations générales, importation des données, informations sur un individu (abeille), informations sur un groupe d'individus
- L'application doit être robuste et ne pas comporter de bug
- L'application doit permettre à un utilisateur d'importer un fichier de données csv ou xlsx dans un format spécifique.
- L'application doit être interactive et performante
- L'application doit permettre de prévisualiser le fichier importé.
- L'application doit permettre d'obtenir différentes données / statistiques pour chaque individu (abeille) d'un dataset
- L'application doit permettre d'obtenir différentes données / statistiques pour un groupe d'individus donné.
- L'application doit permettre de sélectionner à la main des groupes d'individus
- L'application doit pouvoir indiquer clairement les erreurs ou les données manquantes lors de l'importation d'un dataset
- L'application doit pouvoir classer les individus en fonction de leur score d'apprentissage.
- L'application doit permettre de comparer le comportement d'un individu à un ou plusieurs modèles d'apprentissage par renforcement.
- L'application doit permettre de comparer le comportement d'un groupe individus à un ou plusieurs modèles d'apprentissage par renforcement.
- L'application doit permettre de visualiser le résultat d'une expérience pour un individu
- L'application doit permettre d'afficher toutes sortes de graphiques mettant en avant des statistiques intéressantes.
- L'application doit pouvoir classer les modèles d'apprentissage par renforcement selon leur capacité à résoudre le problème ciblé
- L'application doit proposer un suivi en temps réel (Ex : progression d'un calcul quand un modèle s'exécute.)
- L'application doit permettre d'exporter des données sur un individu ou groupe d'individus
- L'application doit permettre d'exporter des graphiques
- L'application doit avoir un style épuré et moderne
- Le code de l'application doit être documenté ou commenté

III.2 Fonctionnalités reportées

III.2.1 Liste des fonctionnalités reportées

- L'application doit posséder une 5ème section principale contenant des informations sur un dataset entier, voir plusieurs datasets cumulés.
- L'application doit permettre d'obtenir différentes données / statistiques pour un dataset donné.
- L'application doit permettre de visualiser la distribution des variables du dataset

III.2.2 Justifications

Comme mentionné dans le dernier rapport, la partie la plus importante de l'application est le recueillement et l'analyse des données pour un individu précis, ainsi que pour un groupe d'individus, le but étant d'analyser en détail leur comportement ainsi que leur stratégie. Après concertation avec les encadrants, il a été déterminé que l'utilité et la manière d'étudier un dataset ou un ensemble de datasets pris entièrement restent moins pertinente. C'est aussi redondant compte tenu de la section "Dashboard". Cependant, cette idée demeure intéressante si l'application sert dans le futur à analyser le comportement d'autres groupes d'individus, voire d'autres espèces d'insectes ou d'animaux.

III.3 Nouvelles perspectives d'améliorations

Sur le long terme, il est souhaitable d'intégrer dans l'application une documentation complète, consultable dans un ou plusieurs onglets des fonctionnalités implémentées. Il s'agirait :

- des modèles utilisés (UCB, LinUCB, Epsilon Greedy, ...)
- des métriques implémentées permettant l'analyse des données (Distance de Wasserstein, Convergence de Kullback-Leibler)
- de l'utilisation en elle-même de l'application
- d'explications concernant le format des données, à l'instar de ce rapport.

De plus, il sera nécessaire d'ajouter de nouveaux modèles d'apprentissage par renforcement, dont la stratégie est plus adaptée pour traiter des problèmes trouvant leur application en biologie. Il est aussi souhaitable d'enrichir l'interface en ajoutant des boutons et des sliders permettant de faire varier des paramètres à la main, ou encore de n'étudier le comportement de l'abeille que sur un certain intervalle de temps (pas seulement de l'essai numéro 1 à 40). En effet, sa stratégie peut se rapprocher d'un modèle au départ, puis d'un autre à la fin.

III.4 Travail réalisé

III.4.1 Liste des tâches techniques effectuées

- Prototype de l'application avec Balsamiq Wireframes (Maintenant dépassé)
- Développement de l'architecture principale (sidebar, menu).
- Développement de l'importation et prévisualisation des données.
- Développement des sections d'analyses individuelles et par groupe.
- Téléchargement des graphiques
- Correction de bugs, Réorganisation du code et optimisation
- Implantation d'un nouveau format d'expérience (plusieurs expériences peuvent être supportées)

III.4.2 Aperçu du prototype initial et de l'application

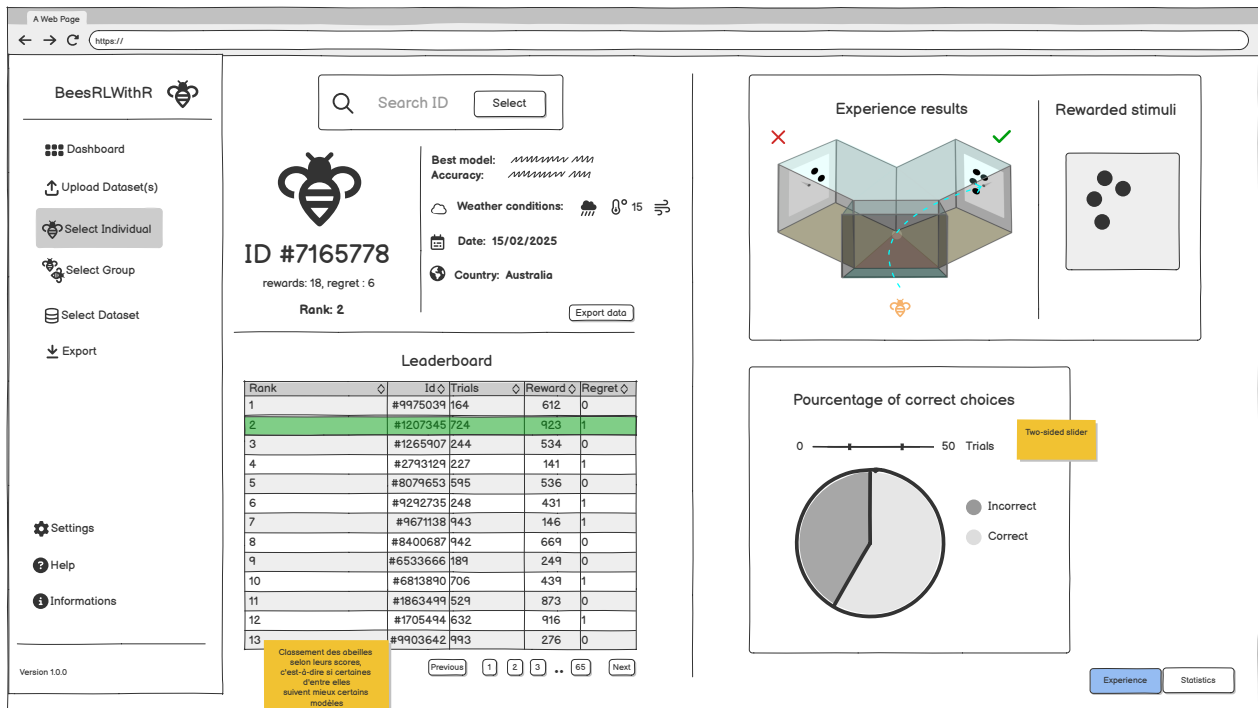


FIGURE III.1 – Aperçu du prototype

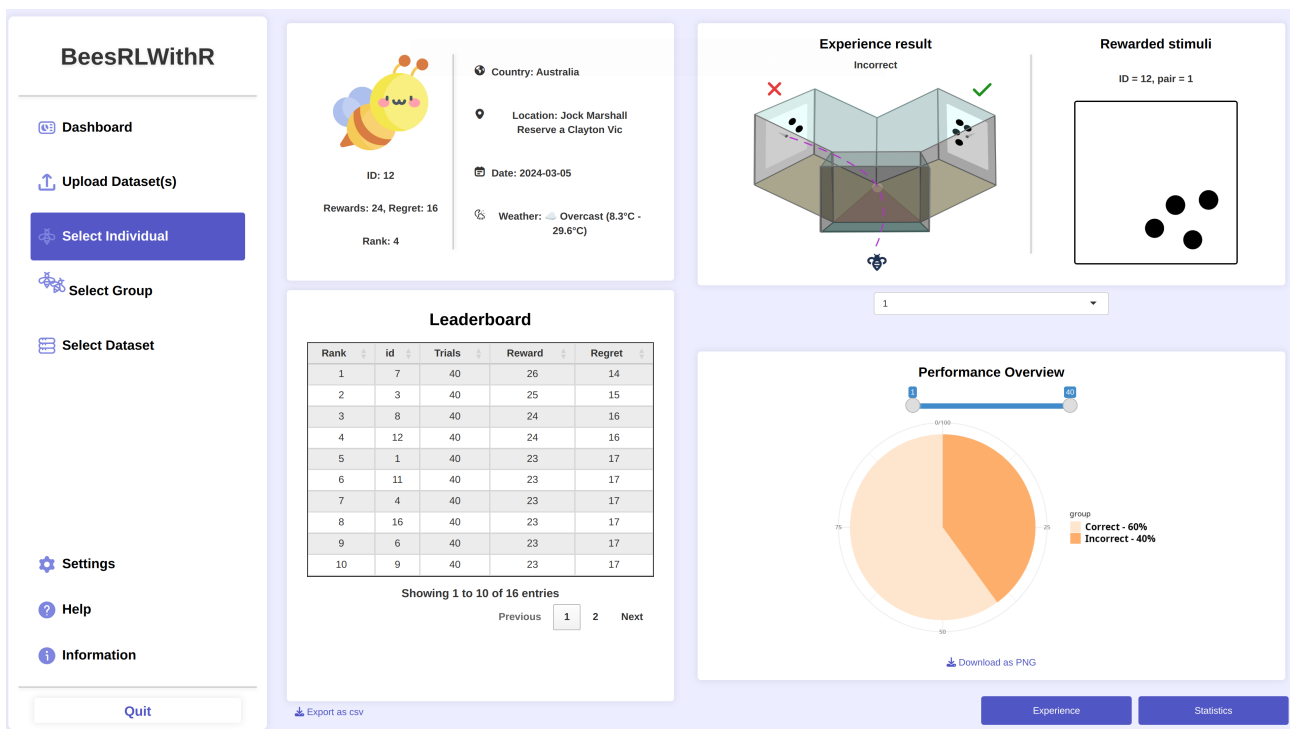


FIGURE III.2 – Aperçu de l'application

IV – Documentation de l'application

IV.1 Structure du projet

IV.1.1 BeesRLWithR

La racine du projet est le dossier `BeesRLWithR` qui contient un fichier `.lintr` pour coder avec VSCode et le fichier `BeesRLWithR.Rproj` pour RStudio. Le code est situé ensuite dans `src/main`.

IV.1.2 `src/main`

Ce dossier contient plusieurs fichiers R :

- Le fichier `app.R` est le plus important. Il suffit de le lancer pour ouvrir l'application dans le navigateur.
- Le fichier `global.R` charge les packages utilisés, load les fichiers sources tels que les modules, et définit quelques variables globales notamment pour les chemins d'accès.
- Les fichiers `server.R` et `ui.R` contiennent l'architecture minimale pour lancer une application shiny et sont appelés par `app.R`.

Il contient aussi des sous-dossiers :

- `R` contient tous les scripts R, aussi bien ceux en lien avec l'affichage graphique que ceux en lien avec le calcul. Nous n'avons pas fait d'architecture MVC (ou ECB) car il n'y a pas de paradigme objet en R. On peut même créer un fichier par fonction. Cela demande cependant une gestion plus fine.
- `modules` contient des modules shiny. Nous reviendrons plus tard dessus. Globalement, les modules sont des éléments encapsulés qui permettent d'ajouter des fonctionnalités spécifiques à l'application pouvant ensuite être réutilisées sans avoir à dupliquer tout le code. C'est ce qui se rapproche le plus des classes.
- `www` est une convention dans les applications Shiny. C'est le dossier dans lequel Shiny va automatiquement chercher les fichiers `style.css` et `script.js`. Généralement, on met aussi dedans toutes les ressources liées au projet comme les données et les images.

IV.1.3 Installation

Attention, certaines dépendances peuvent être problématiques, nécessitant une installation manuelle sur Linux / MacOS / Windows si l'application est utilisée en local avant son déploiement. C'est le cas, par exemple, de la librairie `leaflet` pour l'affichage des cartes interactives. Par oubli de ma part, il sera donc nécessaire de résoudre les problèmes d'installation en regardant sur Internet. De plus, RStudio ne gère pas bien certains graphiques améliorés, comme ceux de la librairie `ggplotly` qui complète `ggplot`.

De manière générale, voici les étapes pour l'installation :

- installer R et Visual Studio Code sur votre machine si ce n'est pas déjà fait. Installer les extensions **R** et **vscodeR**.
- Ne pas oublier dans les paramètres de VSCode de renseigner le chemin d'exécution vers l'exécutable **R**.
- Vérifier le bon fonctionnement de R dans la console puis exécuter pas à pas le fichier `global.R` en essayant de résoudre les bugs d'installations.
- Une fois que tous les packages sont installés, lancer le script `app.R`

IV.2 Notions fondamentales de Shiny utilisées dans le projet

Avant de passer à l'implémentation concrète du projet, nous allons rapidement voir les mécanismes clés sur lesquels repose l'application. En effet, mis à part la syntaxe, c'est surtout la réactivité qui va différencier R des autres langages tels que C, Python ou Java. De plus, il est un peu plus compliqué en R de trouver des ressources pour se former seul dans le but de développer une application complète et donc comprendre les mécanismes graphiques en plus du traitement de données.

IV.2.1 inputs et outputs

Les **inputs** sont généralement des objets définis comme étant des composants graphiques stylisables avec du css, comme en html. Par exemple, un `downloadButton()` ou un `sliderInput()`.

Les **outputs** sont associés à des éléments que l'on veut afficher dans l'interface, mais dont le contenu n'est pas défini tout de suite au moment de l'exécution. C'est par exemple le résultat d'un traitement effectué à partir de la valeur récupérée d'un input. Il s'agit de graphiques, de tableaux ou de textes. En théorie, R nous permet aussi de styliser ces éléments avec du css, même si en pratique, c'est plus compliqué. La stratégie utilisée ici est d'encapsuler ces outputs dans des `div` personnalisées.

Exemple

Admettons que l'on veuille afficher un texte lorsque l'on appuie sur un bouton.

On aura par exemple dans `ui.R`

```
actionButton("generate", "Generer un nombre")
textOutput("random_number")
```

Et dans `server.R`

```
server <- function(input, output) {
  observeEvent(input$generate, {
    output$random_number <- renderText({
      paste("Nombre genere :", sample(1:100, 1))
    })
  })
}
```

On donne l'identifiant `"generate"` au bouton, et `"random_number"` au `textOutput`. Ainsi, on peut ensuite demander de réagir au clic `input$generate` en affichant un texte dans le composant `output$random_number` grâce à `renderText`.

IV.2.2 `reactive()`, `observe()` et `observeEvent()`

Nous n'utilisons pas, pour ceux qui connaissent, de modèle séparant entités, contrôleurs et interfaces graphiques car Shiny repose sur un modèle réactif. On va utiliser des variables qui vont changer d'état selon les actions de l'utilisateur. Ces changements d'état mettront à jour l'interface graphique en utilisant le principe d'inputs / outputs montré ci-dessus.

`reactive()` peut être vu comme une fonction.

- Le bloc `reactive` peut contenir plusieurs lignes de code.
- Une valeur `reactive` peut être appelée dans un contexte réactif avec des parenthèses.

- Cependant, elle attend d'être appelée par un output ou une autre valeur réactive pour réellement s'exécuter.
- Elle ne se met à jour que lorsque ses inputs changent. A ce moment là, elle est recalculée automatiquement.

En résumé :

un réactif est exécuté automatiquement chaque fois qu'un input ou une donnée dont il dépend, change. Il produit une valeur calculée, qui est ensuite utilisée pour des outputs comme des graphiques, des tableaux, etc.

Exemple :

```
df <- reactive({
  req(input$file1) # Verifie qu'un fichier est bien selectionne
  read.csv(input$file1$datapath)
})
```

- Au lancement de l'application, `df()` n'est pas exécuté car `input$file1` est vide.
- Dès que l'utilisateur charge un fichier, `input$file1` change donc `df()` est exécuté.
- Ensuite, si on affiche le dataset (`output$table <- renderDT(df())`), Shiny ne recharge pas `df()` à chaque rendu de l'UI.
- Si l'utilisateur sélectionne un autre fichier, `input$file1` change et Shiny relance `df()`.

`observe()` fonctionne de la même manière. Il exécute un bloc de code mais ne produit pas de valeur. Il est exécuté automatiquement chaque fois qu'un input ou une donnée de laquelle il dépend change.

```
observe({
  req(input$file1)
  print("Nouveau fichier charge !")
})
```

`observeEvent()` fonctionne comme `observe()` mais il réagit à un événement spécifique comme un clic sur un bouton. Il est utilisé pour réaliser une action. Il faut lui passer l'identifiant du composant visé.

```
observeEvent(input$generate, {
  random_num <- sample(1:100, 1)
  output$text <- renderText({paste("Le nombre genere est :", random_num)})
})
```

IV.2.3 Les modules

Ils sont utilisés pour encapsuler des traitements sur des éléments qui peuvent être réutilisés à plusieurs endroits de l'application. En conséquence, ils servent aussi à factoriser du code, et éviter de réécrire plusieurs fois la même logique. La spécificité d'un module est qu'il donnera automatiquement un identifiant différent à chaque élément traité, bien que ces éléments appartiendront à la même "classe". Cependant, j'ai trouvé leur utilisation peu intuitive contrairement à l'approche classique orientée objet d'autres langages. En conséquence, un module n'est utilisé ici que pour encapsuler des éléments graphiques qui ne sont pas des composants de base, et qui changent d'état en fonction des inputs : une table, un graphique de récompenses ou encore un affichage des erreurs au chargement d'un fichier de données. Une amélioration pourrait être effectuée de ce point de vue là. Les modules

ont un format spécifique qu'il faut respecter systématiquement (Voir le code ci-dessous)

Exemple pour le rendu d'une table

```
# Calcul du "namespace" associe au composant courant de la
# "classe" "table" et rendu de la table.
modTableUI <- function(id) {
  ns <- NS(id)
  DTOutput(ns("table"))
}
```

Après l'appel à `modTableUI` avec l'id `tableUpload`, cette table aura l'id final `tableUpload-table` dans l'inspecteur html (F12) du navigateur.

```
modTableServer <- function(id, df, rowHeight, selection) {
  moduleServer(id, function(input, output, session) {

    table_height <- reactiveVal(400)

    observeEvent(session$clientData[[paste0("output_", id, "_height")]], {
      table_height(session$clientData[[paste0("output_", id, "_height")]])
    })

    output$table <- renderDT({
      req(df())
      headerHeight <- 90

      maxRows <- max(floor((table_height() - headerHeight) / rowHeight), 10)

      datatable(df(), options = list(
        paging = TRUE,
        pageLength = maxRows,
        autoWidth = TRUE,
        scrollY = FALSE,
        scrollCollapse = TRUE,
        dom = "Bfrtip"
      ),
      selection = selection,
      class = "display compact cell-border stripe")
    })

    selectedRow <- reactiveVal(NULL)

    observeEvent(input$table_rows_selected, {
      selectedRow(input$table_rows_selected)
    })

    return (selectedRow)
  })
}
```

Dans `modTableServer`, le paramètre `id` est obligatoire.
L'en-tête `moduleServer(id, function(input, output, session))` ne doit jamais changer.

Je peux ensuite appeler mon module à plusieurs endroits dans l'arborescence des composants, mais toutes les tables auront des identifiants différents :


```
div(class = "main-section", id = "upload-preview-section",
  h3(id = "upload-filename", textOutput("upload_filename")),
  div(class = "content-div", id = "preview-dataset",
    modTableUI("tableUpload"))
)
```

Ainsi, je peux récupérer grâce au côté serveur les informations qui m'intéressent. Ici, la variable `selectedRow` va me permettre de connaître l'identifiant de l'abeille sur laquelle l'utilisateur a cliqué dans le leaderboard (une table qui classe les abeilles selon leur score de récompense).

```
selectedRow <- modTableServer(
  "indiv_leaderboard",
  df = reactive({getCurrentLeaderboard(appState)}),
  rowHeight = 40, selection = "single"
)

selectIndividualID(selectedRow, appState)
```

IV.3 Commentaires sur l'implémentation

Dans l'absolu, l'application n'utilise aucun autre mécanisme spécifique à R que ce qui a été présenté dans la section précédente. Le reste découle de la logique, de "bonnes pratiques" de programmation, ainsi que de l'apprentissage du langage pour traiter les datasets, centraliser les éléments réactifs, etc...

IV.3.1 Séparation des fonctionnalités

Pour le développement, nous utilisons une approche incrémentale basée sur le prototype dont un aperçu est disponible plus haut [III.1](#). Ce dernier s'avère très utile. En effet, naturellement, nous commençons par développer la partie importation des données (deuxième onglet : `Upload Dataset(s)`) afin de pouvoir effectuer des traitements sur les données en entrée. Puis pour les besoins du projet, les parties "analyse d'un individu" (troisième onglet `Select Individual`) et "analyse d'un groupe d'individus" (quatrième onglet `Select Group`) sont prioritaires. Nous avons débuté par la partie "individu" dont découle presque directement la partie "groupe".

IV.3.1.1 ui.R

Dans le code, la séparation des onglets se voit facilement. Cette partie du fichier `ui.R` reprend clairement la structure du prototype avec un **ensemble de boutons** dans une **sidebar**, et un **contenu principal** en dessous. Avec le css et un `display: flex;`, ce contenu sera sur la droite.

```
tags$body(
  div(class = "sidebar",
    h2("BeesRLWithR", class = "sidebar-title"),
    div(class = "sidebar-buttons",
      actionButton("dashboard", "Dashboard", class = "sidebar-button"),
      actionButton("upload", "Upload Dataset(s)", class = "sidebar-button"),
      actionButton("individual", "Select Individual", class = "sidebar-button"),
      actionButton("group", "Select Group", class = "sidebar-button"),
      actionButton("dataset", "Select Dataset", class = "sidebar-button"),
      actionButton("export", "Export", class = "sidebar-button"),
      div(class = "utils-buttons",
        actionButton("settings", "Settings", class = "sidebar-button"),
        actionButton("help", "Help", class = "sidebar-button"),
        actionButton("info", "Information", class = "sidebar-button")
      )
    ),
    actionButton("quit", "Quit", class="quit-button")
  ),
  div(class = "main-content",
    uiOutput("main_content")
  )
)
```

IV.3.1.2 server.R

Du côté serveur, on écoute les événements de clic à la souris sur chacun de ces boutons et l'affichage est modifié en conséquence. Si par exemple l'utilisateur appuie sur le bouton `input$individual`, alors la fonction `renderIndividualsContent()` sera appelée. Il y a un fichier `render{Section}Content()` pour chaque section.

```
server <- function(input, output) {

  appState <- createAppState()
  ...
  observeEvent(input$quit, {
    stopApp()
  })
  ...
  observeEvent(input$individual, {
    output$main_content <- renderUI({ renderIndividualsContent() })
  })
  ...
  handleIndividualsEvents(input, output, appState)
}
```

Cependant, l'interface graphique d'une section est réactive. En conséquence, ces fichiers de type "render" ne contiennent pas seulement l'arborescence principale "html" de la section car certains composants dépendent de paramètres dynamiques qui ne peuvent pas être passés par le serveur. Par exemple, pour afficher l'image du *Maze* de l'expérience indiquant si une abeille est allée du bon côté ou non, il faut disposer d'un identifiant d'abeille (ici, le paramètre `selectedTrialData` correspond à la ligne du dataset concernant l'abeille couramment sélectionnée dans le leaderboard) :

```
displayMaze <- function(output, selectedTrialData) {
  output$indiv_maze_result <- renderUI({
    data <- selectedTrialData()

    if (data$correct == 1) {
      tagList(
        div(class = "info-text", paste0("Correct")),
        img(src = "/images/Maze-correct.png")
      )
    } else {
      tagList(
        div(class = "info-text", paste0("Incorrect")),
        img(src = "/images/Maze-incorrect.png")
      )
    }
  })
}
```

C'est pour cela qu'à chaque fichier de rendu est associé un fichier de type **"handlers"**. Un handler associé à une page spécifique de l'application ne modifie **que des données et des composants appartenant à cette page**. La seule exception concerne certaines valeurs réactives. De plus, la fonction de type "handler" est un "main" (programme principal) qui définit ses propres variables et utilise ses propres fonctions. Elle communique également avec l'interface de rendu :

```
handleIndividualsEvents(input, output, appState)
```

Exemple pour `handleIndividualsEvents`

```
handleIndividualsEvents <- function(input, output, appState) {

  metrics <- getSelectedMetrics(input)
  metricsMethods <- getSelectedMetricsMethods(input)

  displayInfos(output, appState)
  displayPage(input, output, appState)
  updatePageState(input, output, appState)

  selectedRow <- modTableServer(
    "indiv_leaderboard",
    df = reactive({getCurrentLeaderboard(appState)}),
    rowHeight = 40, selection = "single"
  )

  selectIndividualID(selectedRow, appState)
  updateGraphs(appState, metrics, metricsMethods)
  addModelComparison(input, appState)

  ...
}
```

```

selectIndividualID <- function(selectedRow, appState) {
  observe({
    req(selectedRow())

    if (!is.null(selectedRow()) && length(selectedRow()) > 0) {
      df <- getCurrentLeaderboard(appState)
      newSelectedId <- df[selectedRow(), "id"]
      selectIds(appState, newSelectedId)
    } else {
      removeSelectedIds(appState)
    }
  })
}

```

IV.3.1.3 Fichiers autres

Voici une liste de fichiers supplémentaires qui servent à séparer les différentes logiques :

- `datasetsEncoding.R` pour gérer le nettoyage des données
- `datasetsFormat.R` pour vérifier le format des données
- `files.R` pour gérer le chargement des fichiers
- `stats.R` qui implémente les différents calculs de probabilité / statistique visualisés sur des graphiques et permettant l'analyse.
- `model.R` qui encapsule les fonctions d'apprentissage par renforcement et fait le lien avec la bibliothèque "banditWithR"
- ...

IV.3.2 Les environnements

La majeure partie de l'application repose sur des "environnements". Un environnement peut être vu comme une structure de données qui à une variable associe un ensemble **unique** de fonctions et d'autres variables. L'appartenance d'une variable à l'une des composantes d'un environnement est déterminée grâce à au moins une des caractéristiques de cette variable : le type, la valeur, le nom d'une colonne dans un dataset... Les implémentations les plus simples d'un environnement sont les `map` en Python et en quelque sorte les `interfaces` en Java. L'implémentation et l'utilisation voulues ici sont similaires à celles des **environnements d'exécutions** et des **analyseurs syntaxiques** en OCaml.

Dans le fichier `environments.R` est défini l'environnement `appState`, le plus important :

```

createAppState <- function() {
  list(
    uploaded = list(
      files = reactiveVal(list()), # Noms des fichiers
      datasets = reactiveValues(), # Donnees contenues fichier (raw data, cleaned
        data, excel sheets, etc.)
      bees = reactiveVal(list()) # Ids des abeilles du dataset, utile uniquement
        pour l'affichage
    ),
    selection = list(
      file = reactiveVal(NULL), # Nom du fichier actuellement selectionne
      individualSelectedId = reactiveVal(NULL), # Id de l'individu selectionne
        dans la section 'Select Individual'
      selectedGroup = reactiveVal(NULL), # Group actuellement selectionne
      groupStorage = reactiveVal(list()), # Ensemble des groupes crees par
        l'utilisateur. Un groupe est compos d'un nom
        # et de la liste des Ids des abeilles
        qui en font partie
      editedGroup = reactiveVal(NULL), # Group en cours de modification
      sheets = reactiveVal(NULL), # Il s'agit de l'ensemble des feuilles
        disponibles dans le dataset selectionne
        # Si le fichier n'est pas un fichier
        excel, ce sera juste la table
      leaderboard = reactiveVal(NULL), # Leaderboard des abeilles associ au
        fichier selectionne
      experimentType = reactiveVal(NULL) # Type d'experience d'ect . C'est
        une chaine de caracteres
    ),
    exports = list(
      regGraphReactive = reactiveVal(), # Garde la trace du graphique de regrets
        consulte par l'utilisateur
      divGraphReactive = reactiveVal(), # Garde la trace de du graphique de
        divergences consulte par l'utilisateur
      pieChartReactive = reactiveVal() # Garde la trace du pie chart consulte par
        l'utilisateur
    ),
    model = list(
      active = reactiveVal(defaultModels), # Liste des modeles selectionnes ou
        actifs
      resultsDataset = reactiveVal() # C'est l'ensemble des resultats des calculs
        de chaque modele de RL
        # pour chaque abeille stocke dans un
        structure de donnee afin de ne pas
        effectuer
        # plusieurs fois un long calcul pour charger
        des graphes par exemple
    )
  )
}

```

La variable associée est globale :

```
appState <- createAppState()
```

Cet environnement remplit plutôt le rôle d'une classe. Il est implémenté par une liste de listes. Il possède plusieurs avantages :

- Les valeurs réactives présentes dans cette structure doivent être accessible partout, cela évite donc de passer 50 paramètres aux fonctions.
- Nous regroupons toutes les valeurs "globales" dans un seul objet logique, un peu comme un état partagé. La synchronisation est donc plus simple.
- C'est lisible, maintenable, scalable.

- Il nous permet d'ajouter une logique de mise à jour centralisée.

Nous pouvons ainsi définir des opérations sur cet environnement.

Ajouter un fichier à la liste des fichiers uploadés

```
addUploadedFile <- function(appState, data) {
  appState$uploaded$files(append(appState$uploaded$files(), data$name))
  addDataset(appState, data)
}
```

Récupérer la liste des fichiers uploadés

```
getUploadedFiles <- function(appState) {
  return (appState$uploaded$files())
}
```

Supprimer un fichier de la liste des fichiers uploadés

```
removeUploadedFile <- function(appState, filename) {
  appState$uploaded$files(setdiff(appState$uploaded$files(), filename))
  removeDataset(appState, filename)
  removeSelectedSheets(appState)
  removeSelectedFile(appState)
  removeCurrentExperiment(appState)
  removeCurrentLeaderboard(appState)
}
```

En effet, si on supprime un fichier, il est sans doute préférable que les autres données associées à ce fichier soient également supprimées, bien que ce choix soit discutable (il est envisagé de bufferiser une partie des informations utiles).

En conclusion, cet environnement sera utilisé pour gérer la réactivité entre les différentes sections et fonctionnalités. La variable `appState` sera passée presque partout en paramètres.

IV.3.3 Importation des données

Dans cette section, nous allons nous intéresser à l'implémentation concrète du chapitre traitant du format de l'expérience et des données : II. Cette section est très importante pour plusieurs raisons :

- C'est la source principale de bugs puisque les fichiers sont dans un premier temps **extérieurs** à l'application. Pour les fichiers Excel qui sont supportés, les reformater, c'est les **parser**, en **entier**.
- La réalité des besoins du projet implique la nécessité de supporter d'autres types d'expériences que celle présentée dans l'introduction. Cependant, le principe reste le même. Ainsi, c'est la partie qui est amenée à **changer** le plus dans le temps si le projet dure.
- Si le fichier n'arrive pas à être loadé, on ne peut rien faire des données qu'il contient.

Comme cette partie n'est cependant pas très compliquée, je conseille de la consulter entièrement. Globalement, il s'agit des fichiers `files.R`, `renderUpload.R`, `handleUploadEvents.R`, `datasetsEncoding.R`, `datasetsFormat.R` et des modules `modErrorsUpload.R` ainsi que `modFileUpload.R`.

IV.3.3.1 Lecture d'un fichier

La fonction `readFile` permet de lire un fichier quelque soit son extension. Elle n'est utilisée qu'à un seul endroit, dans le module `modFileUpload` :

```
modFileUploadServer <- function(id) {  
  moduleServer(id, function(input, output, session) {  
  
    getData <- reactive({  
      req(input$file)  
      data <- readFile(input$file)  
      data  
    })  
  
    return (getData)  
  })  
}
```

```
readFile <- function(file) {  
  ext <- tools::file_ext(file$name)  
  raw <- NULL  
  sheets <- NULL  
  type <- NULL  
  
  if (ext == "csv") {  
    raw <- read.csv2(file$datapath, header = TRUE, sep = ',')  
    type <- "csv"  
  } else if (ext == "xlsx") {  
    sheets <- readExcelSheets(file$datapath)  
    raw <- sheets$Training  
    type <- "excel"  
  } else if (ext == "rds") {  
    raw <- readRDS(file$datapath)  
    type <- "rds"  
  } else {  
    stop("Format de fichier non supporte.")  
  }  
  
  expType <- detectExperimentType(raw)  
  
  errors <- verifyFormat(raw, expType)  
  if (errors != "") {  
    return (list(raw = raw, errors = errors))  
  }  
  
  return (  
    list(  
      path = file$datapath, name = file$name, type = type,  
      sheets = sheets, raw = raw, expType = expType  
    ))  
}
```

Cette fonction possède 3 rôles :

- Lire le fichier "csv", "xlsx" ou "rds". "rds" est une extension pour les fichiers objets R sérialisés. Ce format est utilise pour le debugging : on peut par exemple formater un dataset puis le stocker pour effectuer des tests sans jamais avoir à refaire les traitements initiaux.
- Détecter le type de l'expérience. Il y a deux expériences pour le moment : la première décrite dans ce rapport, ainsi qu'une seconde. La seconde repose sur le même principe mais elle contient des individus qui n'ont pas été entraînés en suivant la même méthode. Avec la méthode *Spaced*, une abeille fait un essai, récupère sa récompense, et rentre à la ruche. Avec la méthode *Massed*, elle peut faire plusieurs essais. Ainsi, on peut étudier l'importance du premier essai, ainsi que

la chaîne de récompenses d'une abeille. Aussi, les stimuli sont ici des couleurs.

- Enfin, elle vérifie que les données sont bien formatées pour cette expérience.

IV.3.3.2 Détecter l'expérience

Pour détecter le type de l'expérience, le programme appelle la fonction `detectExperimentType`

```
detectExperimentType <- function(df) {  
  if ("pair" %in% colnames(df)) {  
    return("EXP1")  
  } else if ("bout" %in% colnames(df)) {  
    return("EXP2")  
  } else {  
    return("unknown")  
  }  
}
```

Comme seulement deux types d'expériences sont disponibles pour le moment, la présence de la colonne `pair` dans le dataset suffit à les différencier. Dans le futur, il conviendra de modifier cette fonction.

IV.3.3.3 Vérification du format

La fonction `verifyFormat` prend en paramètres le dataset et le type de l'expérience.

```
# Verifie qu'un dataset est valide en fonction de ses donnees et du type de  
# l'experience  
verifyFormat <- function(rawDataset, experimentType) {  
  errors <- ""  
  env <- getExperimentsEnv()  
  
  if (!(experimentType %in% names(env))) {  
    return("Unknown experiment type")  
  }  
  
  # On recupere l'environnement des fonctions de verification  
  checkFunctions <- env[[experimentType]]$columns  
  
  # On appelle chaque fonction avant de renvoyer la liste des erreurs  
  for (checkFn in checkFunctions) {  
    errors <- checkFn(rawDataset, errors)  
  }  
  
  return(trimws(errors))  
}
```

Son principe est assez simple. Elle récupère d'abord un environnement contenant toutes les expériences, vérifie la présence de l'expérience dans cet environnement puis en récupère une liste de fonctions qui doivent être appelées. Ces fonctions sont très simples et réalisent un traitement atomique comme vérifier la présence de la colonne `id` ou vérifier qu'il n'y a pas de valeurs manquantes de type `NA`.

Environnement des expériences

```
getExperimentsEnv <- function() {
  return(list(
    EXP1 = list(
      contextFunction = function(df) {
        cbind(encodeNumericContext(df, v1 = 4, v2 = 2), encodePairContext(df))
      },
      filterFunction = function(df) { df },
      defaultContext = c(4, 2),
      columns = list(id=checkId, trial=checkTrial, side=checkCorrectSide,
        pair=checkPairContext, correct=checkCorrect)
    ),
    EXP2 = list(
      contextFunction = function(df) {
        encodeNumericContext(df, v1 = 48, v2 = 44)
      },
      filterFunction = function(df) {
        df$realTrial <- df$trial
        df$trial <- df$bout
        df[df$rep == 1, ]
      },
      defaultContext = c(48, 44),
      columns = list(id=checkId, trial=checkTrial, side=checkCorrectSide,
        bout=checkBout,
        correct=checkCorrect, rep=checkRep)
    )
  ))
}
```

La liste de fonctions mentionnée ci-dessus correspond à la variable `columns`. On voit aussi que le contexte utilisé diffère selon les expériences, de même que la manière d'encoder les données dans le dataset brut. Une fonction permet aussi de filtrer les données : nous n'avons pas forcément besoin de toutes les lignes ou informations.

Fonctions `checkId` et `checkTrial`

```
checkId <- function(rawDataset, errors) {
  if (!(("id" %in% colnames(rawDataset)) ||
    !all(suppressWarnings(!is.na(as.numeric(rawDataset$id)))))) {
    errors <- paste(errors, "La colonne 'id' n'a pas ete trouvee ou ne contient
      pas que des nombres.", sep = "\n")
  }
  return (errors)
}

checkTrial <- function(rawDataset, errors) {
  if (!(("trial" %in% colnames(rawDataset)) ||
    !all(suppressWarnings(!is.na(as.numeric(rawDataset$trial)))))) {
    errors <- paste(errors, "La colonne 'trial' n'a pas ete trouvee ou ne
      contient pas que des nombres.", sep = "\n")
  }
  return (errors)
}
```

IV.3.3.4 Traitement initial des données

Cette partie est **importante**. Après l'importation d'un fichier, l'état global de l'application est mis à jour via l'environnement `appState`. Les données subissent tout de suite un traitement. Il s'agit de **calculer en amont et une seule fois par fichier** les résultats de l'application des algorithmes d'apprentissage par renforcement sur les données et de les stocker dans une structure de données. L'API

Shiny restant assez simpliste et peu optimisée pour des applications conséquentes ou nécessitant une meilleure gestion des performances (notamment de la mémoire dans notre cas), les affichages des graphiques étaient devenus trop lents pour l'onglet 'Dashboard'. À titre d'exemple, les regrets cumulés de tous les algorithmes étaient recalculés à chaque fois que leur affichage était nécessaire. Or tant qu'on ne change pas de fichier, ces regrets restent identiques. Les fonctions appelées sont les suivantes :

```
updateModelsResultsDataset <- function(appState) {
  dataset <- getCurrentCleanedDataset(appState)

  appState$model$resultsDataset(
    generateModelsDataset(
      dataset,
      getActiveModels(appState),
      unique(dataset$id),
      FALSE,
      getCurrentExperiment(appState)
    )
  )
}
```

```
generateModelsDataset <- function(df, activeModels, idList, average,
  experimentType) {
  tic()

  modelData <- list()
  modelsEnv <- getActiveModelsEnv(activeModels)

  for (id in idList) {
    dfIndiv <- selectIndividual(df, id)

    visitorReward <- getVisitorReward(dfIndiv)
    choices <- getChoices(dfIndiv)
    context <- encodeAllContext(dfIndiv, experimentType)

    data <- getModelsPredictions(modelsEnv, context, visitorReward, choices,
      average)

    for (modelName in names(data)) {
      modelData[[modelName]][[as.character(id)]] <- list(
        results = data[[modelName]]$results,
        rewards = data[[modelName]]$reward,
        regrets = data[[modelName]]$regret
      )
    }
  }

  time <- toc()

  print(paste("Temps de generation des resultats : ", time$toc - time$tic))

  return (modelData)
}
```

Analysons un peu la fonction `generateModelsDataset`. Elle prend en paramètre le dataset, les modèles et la liste des individus sélectionnés, ainsi que le type de l'expérience. L'utilisateur peut par exemple vouloir analyser la courbe des regrets du groupe formé par les abeilles 1, 4, 5 et 11, et la comparer avec celles prédites par UCB et LinUCB.

La fonction `getActiveModelsEnv` renvoie, parmi les modèles actifs, la liste des fonctions à appeler. En particulier, elle utilise cet environnement :

```
getModelsEnv <- function() {
  return (list(
    UCB = UCBStrategy,
    UNIFORM = UniformBanditStrategy,
    EPSILONGREEDY = EpsilonGreedyStrategy,
    LINUCB = LinUCBStrategy,
    INDIVIDUAL = IndividualStrategy
  ))
}
```

La fonction `getModelsPredictions` construit une liste qui à chaque modèle, associe les résultats de la fonction appelée. Un résultat est aussi une liste contenant le regret cumulé, la récompense cumulée, la liste des choix...

```
getModelsPredictions <- function(modelsEnv, context, visitorReward, choices,
  average=FALSE) {
  acc <- list()

  for (modelName in names(modelsEnv)) {
    fun <- modelsEnv[[modelName]]

    if (modelName == "LINUCB") {
      result <- fun(context, visitorReward, average)
    } else if (modelName == "INDIVIDUAL") {
      result <- fun(visitorReward, choices, average)
    } else {
      result <- fun(visitorReward, average)
    }

    acc[[modelName]] <- result
  }

  return (acc)
}
```

Enfin, la fonction `generateModelsDataset` construit une liste contenant pour chaque modèle, les résultats précédents pour chaque abeille. La structure résultante est donc une liste de dimension 3. Pour résumer, on a une liste de modèles et chaque modèle stocke les résultats des algorithmes (prédictions) pour chaque abeille. Cela nous amène à expliciter comment les modèles prédisent ces résultats ensuite utilisés pour la visualisation.

IV.3.4 Définition des modèles

Ci-dessous les fonctions associées aux modèles UCB et LinUCB. Ce sont des **wrapper** des méthodes de la librairie *banditWithR*

```

UCBStrategy <- function(visitorReward, average) {
  alloc <- UcbBanditObjectEvaluation(visitor_reward = visitorReward, alpha = 1,
                                     average = average, IsRewardAreBoolean =
                                     FALSE)

  data <- alloc$ucb_alloc

  return (list(results=data$S, choices=data$choice, playingProba=data$proba,
               reward=getCumulativeReward(data$choice, visitorReward),
               regret=alloc$cum_reg_ucb_alloc))
}

LinUCBStrategy <- function(context, visitorReward, average) {
  alloc <- LinucbBanditObjectEvaluation(
    dt = context,
    visitor_reward = visitorReward,
    alpha = 1,
    K = ncol(visitorReward),
    average = average,
    IsRewardAreBoolean = FALSE
  )

  data <- alloc$linucb_bandit_alloc

  S <- getSMatrix(data$choice, visitorReward)

  return (list(results=S, choices=data$choice, playingProba=data$proba,
               reward=getCumulativeReward(data$choice, visitorReward),
               regret=alloc$cum_reg_linucb_bandit_alloc))
}

```

Ces fonctions prennent toutes en paramètre un objet de type **dataframe** appelé `visitorReward`. Cet objet contient tout simplement pour une abeille donnée, la récompense obtenue encodée à l'aide des valeurs du contexte (par exemple **4** et **2** pour l'expérience 1), et ce pour chaque essai. En clair, la récompense obtenue par une abeille pour l'essai `i` est égale à `visitorReward[i, choices[i]]`, soit la valeur contenue dans la colonne `choices[i]` (gauche ou droite) à la ligne `i`.

Systématiquement, les variables `alloc` et `data` vont contenir les résultats réels de l'algorithme d'apprentissage par renforcement. Par exemple, voici à quoi ressemble `LinucbBanditObjectEvaluation` :

```

function (dt, visitor_reward, alpha = 1, K = ncol(visitor_reward),
         average = FALSE, IsRewardAreBoolean = FALSE, dt.reward = dt,
         explanatory_variable = colnames(dt.reward))
{
  linucb_bandit_alloc <- LINUCB(dt = dt, visitor_reward = visitor_reward,
                                alpha = alpha, K = K)
  if (average == FALSE)
    cum_reg_linucb_bandit_alloc <- cumulativeRegret(linucb_bandit_alloc$choice,
                                                    visitor_reward)
  if (average == TRUE)
    cum_reg_linucb_bandit_alloc <-
      cumulativeRegretAverage(linucb_bandit_alloc$choice,
                              visitor_reward = visitor_reward, dt = dt.reward,
                              IsRewardAreBoolean = IsRewardAreBoolean, explanatory_variable =
                              explanatory_variable)
  return(list(linucb_bandit_alloc = linucb_bandit_alloc,
              cum_reg_linucb_bandit_alloc = cum_reg_linucb_bandit_alloc))
}

```

Ici, `linucb_bandit_alloc` est une liste contenant les choix pris par l'algorithme, la probabilité avec laquelle chaque bras a été joué, ou encore le temps d'exécution.

Finalement, on retourne les choix (numéro du bras à chaque essai), la récompense cumulée au cours

du temps (une liste de valeurs), le regret cumulé ainsi qu'une matrice `S`. Cette dernière contient la récompense moyenne apportée par chaque bras, avec le nombre de fois qu'il a été joué.

Ces fonctions avec le stockage des résultats dans l'état global de l'application permettent une mise à jour **facile** et **efficace** de l'interface graphique.

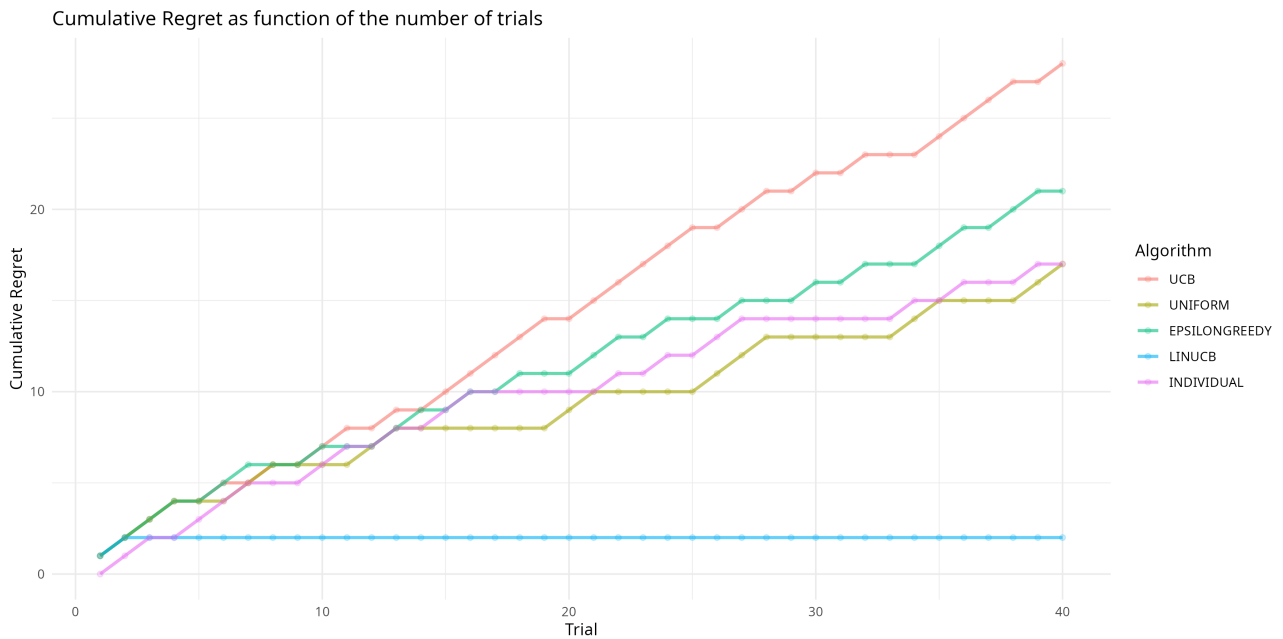


FIGURE IV.1 – Graphique affichant le regret cumulé en fonction du temps

IV.3.5 Définition des tests statistiques

Outre le regret, d'autres indicateurs permettent d'étudier en détail le comportement d'une abeille ainsi que la qualité de l'apprentissage. Des **distances statistiques** peuvent ainsi mesurer à quel point la stratégie d'une abeille diffère de celle employée par un modèle d'apprentissage par renforcement. À noter que nous pouvons aussi comparer deux modèles entre eux. Cela permet d'obtenir des graphiques plus parlants, pour lesquels l'analyse est plus directe. Cependant, leur utilisation est davantage justifiée pour des algorithmes qui produisent des distributions de probabilités autour des bras. Pour l'instant, ces distances ne sont implémentées que pour **UCB** et **LinUCB**.

On rappelle qu'une abeille explore différents bras dans le but de maximiser sa récompense et minimiser son regret. Au départ, elle n'a pas connaissance du meilleur bras. Au fil du temps, elle élabore une stratégie et construit une **croyance** sur la qualité de chaque bras. C'est sa **distribution de probabilité**. Maintenant, on veut comparer cette croyance à une autre, plus experte. En d'autres termes, on veut savoir **à quel point la stratégie de l'abeille diffère d'une stratégie optimale**.

IV.3.5.1 Divergence de Kullback-Leibler

La divergence de Kullback-Leibler (ou divergence KL), selon *Wikipédia*, mesure la **dissimilarité** entre une distribution de probabilité P (calculée de manière optimale sur les données ou les observations) et une distribution de probabilité Q (un modèle expérimental ou une approximation de P). Elle donne la quantité moyenne d'information obtenue (au sens de Shannon) si l'on utilise Q à la place de P , lorsque P est la vraie distribution.

En considérant que P et Q sont des distributions discrètes sur un ensemble X d'essais, la divergence KL est définie par la formule suivante :

$$D_{\text{KL}}(P\|Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$$

Si $P(x) \approx Q(x)$ **pour tout** x , alors $\frac{P(x)}{Q(x)} \approx 1$ donc $\log \frac{P(x)}{Q(x)} \approx 0$. Finalement, la somme entière tend vers 0 et la courbe tend vers 0. Cela implique que **l'abeille reproduit parfaitement la stratégie du modèle**.

Si $Q(x)$ **est très différent de** $P(x)$, alors le rapport $\frac{P(x)}{Q(x)}$ sera **très grand ou très petit**, le logarithme sera **plus élevé** et donc la somme va augmenter. En résumé, plus le KL est grand, plus **le comportement de l'abeille est loin de celui attendu**.

Globalement, une décroissance de la courbe peut traduire un processus d'apprentissage tandis que de fortes fluctuations ou une croissance traduisent une dissimilarité dans le comportement.

IV.3.5.2 Distance de Wasserstein

Là où la divergence KL mesure un écart entre deux distributions de probabilités, la distance de Wasserstein (aussi appelée *Earth Mover's Distance*) représente le coût minimum pour "déplacer" une distribution vers l'autre, c'est-à-dire transformer Q en P . On pourrait la voir comme "Quel effort l'abeille doit-elle fournir pour modifier sa stratégie, afin que ses choix deviennent corrects?".

On voit quand même que l'interprétation est moins naturelle. Il est alors conseillé de lire la section **Intuition et lien avec le transport optimal** de la page Wikipédia que vous trouverez en annexe [10](#).

Dans ce projet, pour avoir plus de contrôle sur les résultats et l'utilisation des probabilités, nous avons utilisé une approximation simplifiée de la distance de Wasserstein 1D (Wasserstein-1). Cette dernière semble suffisante puisque qu'on obtient les mêmes résultats qu'en utilisant la librairie `transport` sur deux vecteurs de probabilités aléatoires.

En considérant les mêmes distributions de probabilités P et Q , on notera csP et csQ les vecteurs

sommes cumulées de ces distributions, c'est-à-dire $csP = [P_0, P_0 + P_1, P_0 + P_1 + P_2, \dots, \sum_{k=0}^K P_k]$, où P_k est la probabilité de choisir le bras k .

On a alors :

$$D_{WD}(P||Q) = \sum_{x \in X} |csP(x) - csQ(x)|$$

En résumé, si la distance de Wasserstein est **petite**, cela signifie que l'abeille est **proche** de la bonne réponse, elle n'a besoin de faire que de petits ajustements. Si la distance est **grande**, alors elle est loin des choix optimaux, sa stratégie doit être modifiée si elle veut apprendre à résoudre le problème.

IV.3.5.3 Implémentations de la divergence KL et de la distance de Wasserstein

Dans l'application, la fonction `getUserDivergence` s'occupe globalement de ces calculs :

```
getUserDivergence <- function(df, id, usedModels, modelsList, method,
  experimentType) {
  dfUser <- df[df$id == id, ]
  if (nrow(dfUser) == 0) return()

  visitorReward <- getVisitorReward(dfUser)
  context <- encodeAllContext(dfUser, experimentType)
  K <- ncol(visitorReward)
  choices <- getChoices(dfUser)

  acc <- getModelsPredictions(usedModels, context, visitorReward, choices)

  P <- getProbaDistributionOverTime(acc[[modelsList[1]]]$choices,
    visitorReward, K)
  Q <- getProbaDistributionOverTime(acc[[modelsList[2]]]$choices,
    visitorReward, K)

  divergence <- getModelDivergenceOverTime(P, Q, method)

  return (divergence)
}
```

Elle prend en paramètres :

- `df` les données
- `id` l'identifiant de l'abeille
- `usedModels` la liste des deux modèles comparés (par exemple UCB et LinUCB, ou bien la stratégie de l'abeille avec UCB).
- `modelsList` la liste des modèles supportés (ceux qui définissent des distributions de probabilités)
- `method` Le type de distance (KL ou WD)
- `experimentType` Le type de l'expérience

Elle utilise aussi la fonction `getProbaDistributionOverTime` qui va construire pour chaque distribution une matrice stockant les résultats en fonction du temps (du nombre d'essais), afin de faciliter et donner plus de sens à la visualisation. Chaque matrice contient n lignes où n est le nombre d'essais, et k colonnes où k est le nombre de bras (ici deux). Ainsi, on connaît à chaque instant la probabilité que le modèle choisisse plutôt un bras ou l'autre.

On s'appuie ensuite sur l'implémentation des distances statistiques :

```

getKLDivergence <- function(P, Q, epsilon=1e-10) {
  P <- P + epsilon
  Q <- Q + epsilon
  return (sum(P * log2(P / Q)))
}

getKLDivergenceOverTime <- function(P, Q, epsilon=1e-10) {
  n <- nrow(P)
  divergences <- rep(0, n)
  for (i in 1:n) {
    divergences[i] <- getKLDivergence(P[i, ], Q[i, ], epsilon)
  }
  return (divergences)
}

getWassersteinDistance <- function(P, Q) {
  cdfP <- cumsum(P)
  cdfQ <- cumsum(Q)
  return(sum(abs(cdfP - cdfQ)))
}

getWassersteinOverTime <- function(P, Q) {
  n <- nrow(P)
  distances <- rep(0, n)
  for (i in 1:n) {
    distances[i] <- getWassersteinDistance(P[i, ], Q[i, ])
  }
  return(distances)
}

```

IV.3.5.4 Visualisation des tests statistiques

Attention, il est nécessaire d'être **prudent** lors de l'analyse de ces métriques. La conversion des données du problème en probabilités force une valeur de divergence variant entre 0 et 1. En conséquence, il n'est pas rare d'observer un écart très faible et la courbe qui tend vers 0. Pour autant, cela ne veut pas dire que l'abeille apprend parfaitement. Il conviendra soit de modifier l'échelle pour accentuer les différences en conservant le sens de la formule (difficile), soit de déplacer le curseur sur la courbe et de prendre du recul sur les résultats.

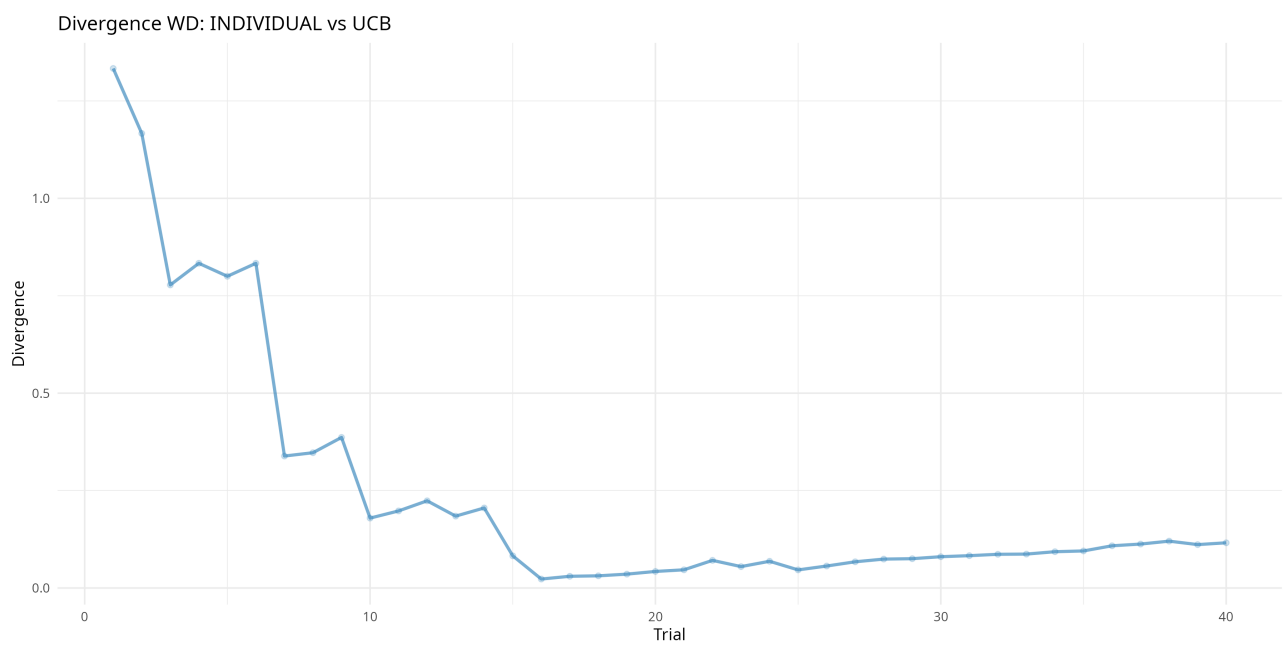


FIGURE IV.2 – Graphique affichant la distance de Wasserstein entre les choix de l'abeille et les prédictions du modèle UCB

V – Contact

Pour tout problème lié à l'installation, à l'utilisation de l'application ou pour n'importe quelle question, merci d'envoyer un mail à l'adresse remi.airiau@gmail.com ou bien Emmanuelle.Claeys@irit.fr

A – Bibliographie

- [1] Shiny - Web Application Development Framework for R. Posit
<https://shiny.posit.co/>
- [2] Shinyapps.io - Website for Shiny Applications Hosting. Posit.
<https://www.shinyapps.io/>
- [3] Claey's Emmanuelle. banditWithR - R Package for Implementing Multi-Armed Bandit Algorithms.
<https://github.com/manuclaeys/banditWithR>
- [4] Airiau Rémi. BeesRLWithR - R Application for Analyzing Bee Decision-Making Process. Github.
<https://github.com/Ravemi987/BeesRLWithR>
- [5] Markov Decision Process. Wikipedia.
https://fr.wikipedia.org/wiki/Processus_de_d%C3%A9cision_markovien
- [6] Multi-Armed Bandit. Wikipedia.
https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_bandit_mancho't
- [7] Multi-Armed Bandit : Data Science Concepts. Youtube.
<https://www.youtube.com/watch?v=e3L4VocZnnQ&t=5s>
- [8] Best Multi-Armed Bandit Strategy ? (feat : UCB Method). Youtube.
<https://www.youtube.com/watch?v=FgmMK6RPU1c&t=3s>
- [9] Kullback–Leibler divergence. Wikipedia.
https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence
- [10] Wasserstein Distance. Wikipedia.
https://fr.wikipedia.org/wiki/Distance_de_Wasserstein
- [11] Claey's Emmanuelle. Introduction to Multi-Armed Bandit. Canva.
https://www.canva.com/design/DAF-LSEEEBw/VcExXyiGMoudF-vL_4p4mA/edit?ui=eyJEljp7IlAiOnsiQiI6ZmFsc2V9fX0

B – Implémentation du cours sur les MDP

```
states <- 1:6
actions <- c("right", "jump", "left", "eat")
P <- list(
  list(c(0, 1, 0, 0, 0, 0), c(1, 0, 0, 0, 0, 0), c(1, 0, 0, 0, 0, 0), c(0, 0, 0, 0, 0, 0)),
  list(c(0, 1, 0, 0, 0, 0), c(0, 0, 1, 0, 0, 0), c(1, 0, 0, 0, 0, 0), c(0, 0, 0, 0, 0, 0)),
  list(c(0, 0, 0, 1, 0, 0), c(0, 0, 1, 0, 0, 0), c(0, 1, 0, 0, 0, 0), c(0, 0, 0, 0, 0, 0)),
  list(c(0, 0, 0, 0, 1, 0), c(0, 0, 0, 1, 0, 0), c(0, 0, 0, 1, 0, 0), c(0, 0, 0, 0, 0, 0)),
  list(c(0, 0, 0, 0, 0, 1), c(0, 0, 0, 0, 1, 0), c(0, 0, 0, 1, 0, 0), c(0, 0, 0, 0, 0, 0)),
  list(c(0, 0, 0, 0, 0, 1), c(0, 0, 0, 0, 0, 1), c(0, 0, 0, 0, 1, 0), c(0, 0, 0, 0, 0, 1))
)

R <- list(
  list(0, -1, -10, 0),
  list(-1, 0, 0, 0),
  list(0, -10, 0, 0),
  list(0, -1, -1, 0),
  list(0, -1, 0, 0),
  list(0, -1, 0, 10)
)
```

```
valueIteration <- function(states, actions, Proba, Reward,
                             gamma = 0.2, epsilon = 1e-4) {
  V <- rep(0, length(states))
  delta <- +Inf

  while(delta > epsilon) {
    delta <- 0
    V_new <- V

    for (s in 1:length(states)) {
      Q_values <- rep(0, length(actions))

      for (a in 1:length(actions)) {
        Q_values[a] <- Reward[[s]][[a]] + gamma * sum(Proba[[s]][[a]] * V)
      }

      V_new[s] <- max(Q_values)
      delta <- max(delta, abs(V_new[s] - V[s]))
    }

    V <- V_new
  }

  return (V)
}
```

```

valueIterationPolicy <- function(states, actions, Proba, Reward,
                                gamma = 0.2, epsilon = 1e-4) {
  V <- rep(0, length(states))
  policy <- rep(NA, length(states))
  delta <- +Inf

  while (delta > epsilon) {
    delta <- 0
    V_new <- V

    for (s in 1:length(states)) {
      Q_values <- rep(0, length(actions))

      for (a in 1:length(actions)) {
        Q_values[a] <- Reward[[s]][[a]] + gamma * sum(Proba[[s]][[a]] * V)
      }

      best_action <- which.max(Q_values)
      V_new[s] <- Q_values[best_action]
      policy[s] <- actions[best_action]

      delta <- max(delta, abs(V_new[s] - V[s]))
    }

    V <- V_new
  }

  return (list(values = V, policy = policy))
}

```

```

policyEvaluation <- function(policy, V, states, actions, Proba, Reward,
                             gamma = 0.2, epsilon = 1e-4) {
  delta <- +Inf
  while (delta > epsilon) {
    delta <- 0
    V_new <- V

    for (s in 1:length(states)) {
      a <- which(policy[s] == actions)[[1]]
      V_new[s] <- Reward[[s]][[a]] + gamma * sum(Proba[[s]][[a]] * V)
      delta <- max(delta, abs(V_new[s] - V[s]))
    }

    V <- V_new
  }

  return (V)
}

```

```

policyImprovement <- function(policy, V, states, actions, Proba, Reward,
                              gamma = 0.2) {
  policy_stable <- TRUE

  for (s in 1:length(states)) {
    Q_values <- rep(0, length(actions))

    for (a in 1:length(actions)) {
      Q_values[a] <- Reward[[s]][[a]] + gamma * sum(Proba[[s]][[a]] * V)
    }

    best_action <- which.max(Q_values)
    if (policy[s] != actions[best_action]) {
      policy_stable <- FALSE
    }

    policy[s] <- actions[best_action]
  }

  return (list(policy = policy, is_stable = policy_stable))
}

```

```

policyIterationGreedy <- function(states, actions, Proba, Reward,
                                  gamma = 0.2, epsilon = 1e-4) {
  V <- rep(0, length(states))

  policy <- sample(actions, length(states), replace = TRUE)
  policy_stable <- FALSE

  while(!policy_stable) {
    V <- policyEvaluation(policy, V, states, actions, Proba, Reward)
    new_policy <- policyImprovement(policy, V, states, actions, Proba, Reward)

    policy <- new_policy$policy
    policy_stable <- new_policy$is_stable
  }

  return (list(values = V, policy = policy))
}

```

```

rewardFunction <- function(state, action) {
  return (R[[state]][[action]])
}

transitionFunction <- function(state, action) {
  return (which.max(P[[state]][[action]]))
}

QLearning <- function(states, actions, TransFun, RewardFun, max_epochs = 100,
                      max_step = 10, gamma = 0.2, alpha = 1, epsilon = 0.1) {

  Q <- matrix(0, nrow = length(states), ncol = length(actions))
  initial_state <- states[1]
  goal_state <- states[length(states)]

  for (epoch in 1:max_epochs) {
    state <- initial_state

    for (step in 1:max_step) {

      if (runif(1) < epsilon) {
        action <- sample(1:length(actions), 1)
      } else {
        action <- which.max(Q[state, ])
      }

      new_state <- TransFun(state, action)
      reward <- RewardFun(state, action)

      Q[state, action] <- Q[state, action] + alpha * (reward + gamma *
        max(Q[new_state, ]) - Q[state, action])

      state <- new_state
    }
  }
  return (Q)
}

```

```

mainMDP <- function() {

  Values <- valueIteration(states, actions, P, R)
  cat(Values, "\n\n")

  valuesPolicy <- valueIterationPolicy(states, actions, P, R)
  cat(valuesPolicy$values, "\n\n")
  cat(valuesPolicy$policy, "\n\n")

  policyGreedy <- policyIterationGreedy(states, actions, P, R)
  cat(policyGreedy$values, "\n\n")
  cat(policyGreedy$policy, "\n\n")

  Qtable <- QLearning(states, actions, transitionFunction, rewardFunction,
    max_epochs = 100, max_step = 10)
  print(Qtable)
}

mainMDP()

```

C – Implémentation des vidéos sur les MAB

```
library(ggplot2)

set.seed(42)
K <- 3
true_means <- c(10, 8, 5)
true_stds <- c(5, 4, 2.5)

rewardFunctionBandit <- function(action) {
  rnorm(1, mean = true_means[action], sd = true_stds[action])
}

epsilonGreedyPrintGraphics <- function(n_iter, K, Q_history) {
  df <- data.frame(
    Iteration = rep(1:n_iter, K),
    Q_Value = as.vector(Q_history),
    Restaurant = factor(rep(1:K, each = n_iter))
  )

  p <- ggplot(df, aes(x = Iteration, y = Q_Value, color = Restaurant)) +
    geom_line() +
    geom_hline(yintercept = true_means, linetype = "dashed", color = "black") +
    labs(title = "Evolution des valeurs estimees des restaurants",
         x = "Iteration",
         y = "Valeur estimee de satisfaction",
         color = "Restaurant") +
    theme_minimal()

  print(p)
}
```

```
epsilonGreedyBandit <- function(K, RewardFun, epsilon = 0.1, n_iter = 300) {
  Q <- rep(0, K)
  N <- rep(0, K)
  total_reward <- 0
  Q_history <- matrix(NA, nrow = n_iter, ncol = K)

  for (t in 1:n_iter) {
    if (runif(1) < epsilon) {
      action <- sample(1:K, 1)
    } else {
      action <- which.max(Q)
    }

    reward <- RewardFun(action)

    N[action] <- N[action] + 1
    Q[action] <- Q[action] + (reward - Q[action]) / N[action]

    total_reward <- total_reward + reward
    Q_history[t, ] <- Q
  }

  epsilonGreedyPrintGraphics(n_iter, K, Q_history)

  return (list(Q = Q, N = N, reward = total_reward))
}
```



```

UCBBandit <- function(K, RewardFun, alpha = 2, n_iter = 300) {
  Q <- rep(0, K)
  N <- rep(0, K)
  total_reward <- 0

  for (t in 1:n_iter) {

    if (any(N == 0)) {
      action <- which(N == 0)[1]
    } else {
      UCB <- Q + alpha * sqrt(2*log(t) / N)
      action <- which.max(UCB)
    }

    reward <- RewardFun(action)

    N[action] <- N[action] + 1
    Q[action] <- Q[action] + (reward - Q[action]) / N[action]

    total_reward <- total_reward + reward
  }

  return (list(Q = Q, N = N, reward = total_reward))
}

```

```

mainMAB <- function() {

  resultEpsilon <- epsilonGreedyBandit(K, rewardFunctionBandit)

  cat("EpsilonGreedy", "\n")
  cat("Estimation finale des recompenses pour chaque bras :", resultEpsilon$Q,
      "\n")
  cat("Nombre de fois que chaque bras a ete joue :", resultEpsilon$N, "\n")
  cat("Recompense totale accumulee :", resultEpsilon$reward, "\n")

  resultUCB <- UCBBandit(K, rewardFunctionBandit)

  cat("UCB", "\n")
  cat("Estimation finale des recompenses pour chaque bras :", resultEpsilon$Q,
      "\n")
  cat("Nombre de fois que chaque bras a ete joue :", resultEpsilon$N, "\n")
  cat("Recompense totale accumulee :", resultEpsilon$reward, "\n")

}

mainMAB()

```